1. **Single Responsibility Principle**

Based on the Cohesion review, there are no classes violating single responsibility principle

2. **Open – Closed Principle**

Module SecurityConfig is having Common coupling issue that may violate the principle, when having another route in WHITE_LIST, developers have to modify the module, which is not so ideal.

Module Regex is having Common coupling issue that violates the principle but as a configuration module, this may be acceptable as only constant regular expressions are put here.

Some service modules, namely CartService, JwtService, ProductService, are having Stamp coupling issue, which may lead to violation of the principles if any changes to the structure of the class User are to be made.

As for improvements, some directions are mentioned in Cohesion and Coupling review report, and with those alternations, these problems are mostly resolved.

3. **Liskov Substitution Principle**

Some custom base classes such as BaseEntity, BaseResponse are defined for generalization and its inherited classes can fully be substitutable for its parent class. All the Entity classes do not conflict with BaseEntity, as for response, at this stage, there has not been anything to judge.

Inheritance trees, organized at this stage, are all 2-generation and have not been found with conflicts.

4. **Interface Segregation Principle**

Interfaces are defined as classes named with the suffix of ‘Repository’, each serves a certain functionality without too many methods in one. There have been interfaces for managing Order, Product, ProductCart, User, WishList and Review(the last two are extended functionality that our group may design and implement)

5. **Dependency Inversion Principle**

Service classes – high level modules, depend on -Repository interfaces – low level modules, through using interface or abstraction rather than concrete implementation.

The structure of communication goes from: Controller – Service – Repository, high level – low level – repository, and current implementation shows that the controller depends directly on service, which violates the principle. The improvement direction is to generalize an interface for controller – service, let the service implements the interface and the controller will depend on the interface.